CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

Student Name: Cian Mac Liatháin **Student ID**: 11476078

Assignment 2

Provide answers to the problems in the boxes provided.

The buttons at the top of the page can be used to **create more boxes if needed**. The type of box can be changed from `Code` to `Markdown`. `Code` boxes take (and execute) `python` code. `Markdown` boxes take (and format nicely) **text input**. In this way, you can provide answers, ask questions, or raise issues, in words.

Some marks will be awarded for participation and engagement.

When finished, print this notebook into a **pdf** file and submit this to **blackboard**.

Deadline is next Monday at 5pm.

Setup

This is a `jupyter` notebook. You can open and interact with the notebook through one of sites recommended at its github repository.

Or, you can install and use `jupyter` as a `python` package on your own laptop or PC.

The following command loads the `networkx` package into the current session.
The next command specifies some standard options that can be useful for drawing graphs.

In order to execute the code in a box, use the mouse or arrow keys to highlight the box and then press SHIFT-RETURN.

In [1]:
```
import networkx as nx
import pandas as pd
from queue import Queue
opts = { "with_labels": True, "node_color": 'y' }
```

Should it ever happen that the notebook becomes unusable, start again with a fresh copy.

1. The Counties of Ireland.

Define a graph `I` on the 32 counties of Ireland by joining two counties whenever they have a common border. (A list of counties and their borders in the adjacency list format can be found on github: check the data and let us know if you disagree!)

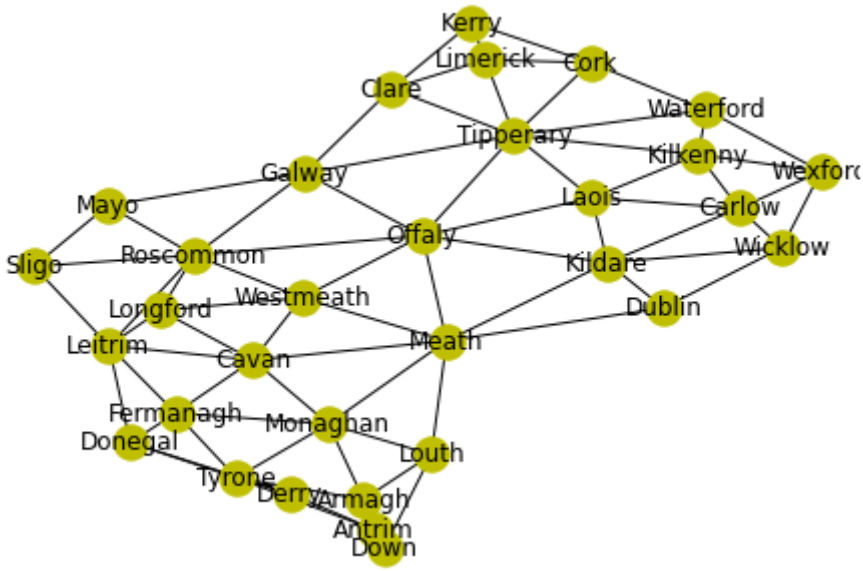
What is the order and the size of the resulting graph?

In terms of centrality measures, what are the 3 most central counties, for

1. degree centrality?
2. eigenvector centrality?
3. closeness centrality?
4. betweenness centrality?

In [2]:
```
#1.0 Read graph data
I = nx.read_adjlist('data/ireland.adj')
```

In [3]:
```
#1.0 Draw graph
nx.draw(I, **opts)
```



In [4]:
```
#1.1 degree centrality
nx.set_node_attributes(I, dict(I.degree()), 'degree')
nx.set_node_attributes(I, nx.degree_centrality(I), '$C_i^D$')

pd.DataFrame.from_dict(dict(I.nodes(data=True)),
                       orient = 'index'
).sort_values('$C_i^D$', ascending=False).iloc[0:3]
```

Out[4]:

	degree	C_i^D
Tipperary	8	0.258065
Roscommon	7	0.225806
Offaly	7	0.225806

In [5]:
```
#1.2 Eigenvector Centrality
eig_cen=nx.eigenvector_centrality(I)
nx.set_node_attributes(I, eig_cen, '$C_i^E$')
pd.DataFrame.from_dict(dict(I.nodes(data=True)),
                       orient = 'index'
).sort_values('$C_i^E$', ascending=False).iloc[0:3]
```

Out[5]:

	degree	C_i^D	C_i^E
Offaly	7	0.225806	0.327506
Meath	7	0.225806	0.289622
Roscommon	7	0.225806	0.266327

In [6]:
```
#1.3 Closeness Centrality
close_cen=nx.closeness_centrality(I)
nx.set_node_attributes(I, close_cen, '$C_i^C$')
pd.DataFrame.from_dict(dict(I.nodes(data=True)),
                       orient = 'index'
).sort_values('$C_i^C$', ascending=False).iloc[0:3]
```

Out[6]:

	degree	C_i^D	C_i^E	C_i^C
Offaly	7	0.225806	0.327506	0.469697
Meath	7	0.225806	0.289622	0.455882
Roscommon	7	0.225806	0.266327	0.413333

In [7]:
```
#1.4 Betweenness Centrality
betw_cen=nx.betweenness_centrality(I)
nx.set_node_attributes(I, betw_cen, '$C_i^B$')
pd.DataFrame.from_dict(dict(I.nodes(data=True)),
                       orient = 'index'
).sort_values('$C_i^B$', ascending=False).iloc[0:3]
```

Out[7]:

	degree	C_i^D	C_i^E	C_i^C	C_i^B
Meath	7	0.225806	0.289622	0.455882	0.308805
Offaly	7	0.225806	0.327506	0.469697	0.279330
Tipperary	8	0.258065	0.258747	0.397436	0.222681

2. Actions on Sets.

Recall the implementation of **permutations as tuples** ...

In [8]:
```
n = 4

def transposition(n, i):
    t = [k for k in range(n)]
    t[i], t[i+1] = t[i+1], t[i]
    return tuple(t)

gens = [transposition(n, k) for k in range(n-1)]
gens
```

Out[8]:
```
[(1, 0, 2, 3), (0, 2, 1, 3), (0, 1, 3, 2)]
```

... and the **orbit algorithm** that creates the **action graph**:

In [9]:
```
def init_Q_G(x):
    Q = Queue()
    Q.put(x)
    G = nx.Graph()
    G.add_node(x)
    return Q, G

def action_graph(gens, x, action, equals):
    Q, G = init_Q_G(x)
    while not Q.empty():
        y = Q.get()
        for a in gens:
            z = action(a, y)
            w = next((v for v in G.nodes() if equals(v, z)), None)
            if w is None:
                Q.put(z)
                G.add_node(z)
                G.add_edge(y, z)
            else:
                G.add_edge(y, w)
    return G
```

Permutations **act on subsets** of $X = \{0, 1, \ldots, n-1\}$ in such a way that, for any subset $Z \subseteq X$ and a permutation f of X, the image of Z under f is determined as

$$f(Z) = \{f(z) : z \in Z\},$$

the **image of the set is the set of the images**. In `python`, using hashable `frozenset`s rather than mutable `set`s, this action can be defined as

In [10]:
```
def on_sets(a, zzz):
    return frozenset(a[z] for z in zzz)
```

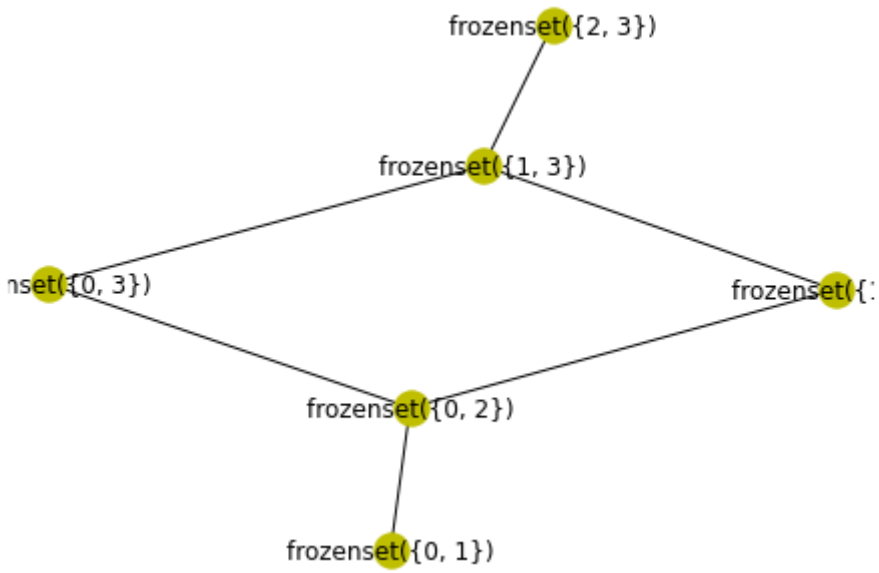
Trusting that equality of (frozen) sets can be tested with `python`'s ordinary equals operator, we again set

In [11]:
```
def eq(x, y):
    return x == y
```

Then we can compute and draw the action graph of the symmetric group S_4 acting on the 2-subsets of $\{0, 1, 2, 3\}$, using $x = \{0, 1\}$ as the initial vertex:

In [12]:

```
x = frozenset({0,1})
G = action_graph(gens, x, on_sets, eq)
nx.draw(G, **opts)
```

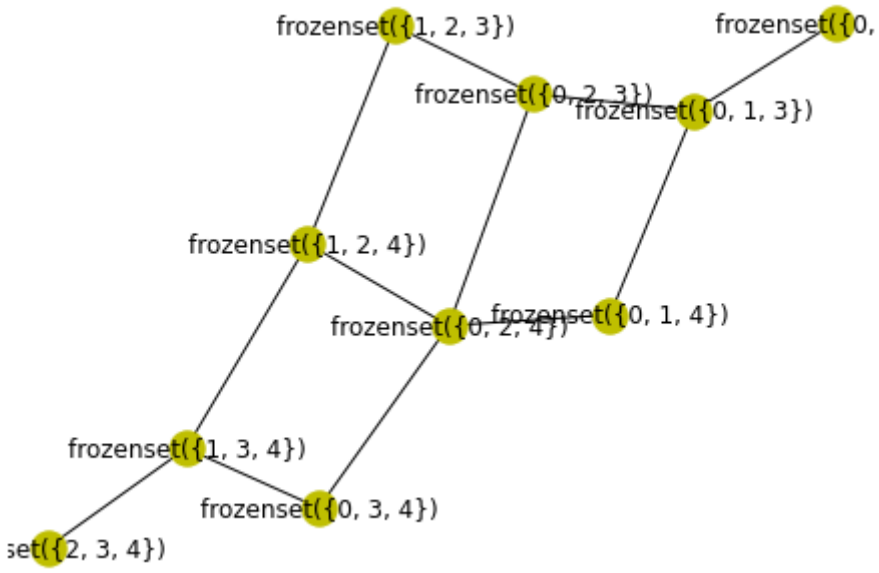


1. Compute and draw the action graph of S_5 acting on the 3**-subsets** of $\{0, 1, 2, 3, 4\}$. (You would need to define a new list `gens` of generators, and find a suitable 3-set `x` to compute the orbit of. Everything else, the `action_graph` function, the action `on_sets` and the equality test `eq` should still work.)

2. Compute and draw the action graph of S_6 acting on the 3-subsets of $\{0, 1, 2, 3, 4, 5\}$.

In [13]:

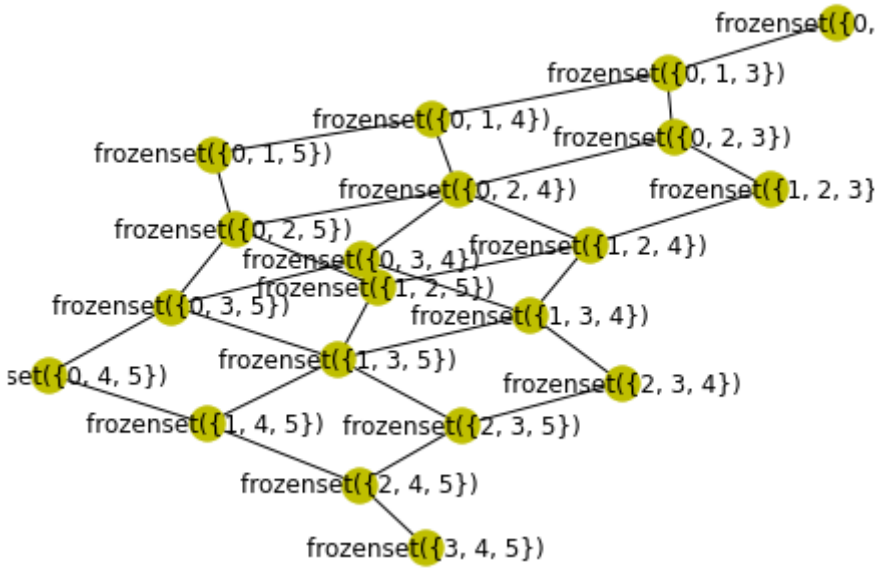
```
#2.1 S5

n = 5
x1 = frozenset({0,1,2})
gens1 = [transposition(n,k) for k in range(n-1)]
G1= action_graph(gens1, x1, on_sets, eq)
nx.draw(G1, **opts)
```



In [14]:
```
G1.nodes()
```

Out[14]:
```
NodeView((frozenset({0, 1, 2}), frozenset({0, 1, 3}), frozenset({0, 2, 3}),
frozenset({0, 1, 4}), frozenset({1, 2, 3}), frozenset({0, 2, 4}), frozenset
({1, 2, 4}), frozenset({0, 3, 4}), frozenset({1, 3, 4}), frozenset({2, 3,
4})))
```

In [15]:
```
#2.2 S6
n=6
gens2=[transposition(n,k) for k in range(n-1)]
G2=action_graph(gens2, x1, on_sets,eq)
nx.draw(G2,**opts)
```



In [16]:
```
G2.nodes()
```

Out[16]:
```
NodeView((frozenset({0, 1, 2}), frozenset({0, 1, 3}), frozenset({0, 2, 3}),
frozenset({0, 1, 4}), frozenset({1, 2, 3}), frozenset({0, 2, 4}), frozenset
({0, 1, 5}), frozenset({1, 2, 4}), frozenset({0, 3, 4}), frozenset({0, 2,
5}), frozenset({1, 3, 4}), frozenset({1, 2, 5}), frozenset({0, 3, 5}), froz
enset({2, 3, 4}), frozenset({1, 3, 5}), frozenset({0, 4, 5}), frozenset({2,
3, 5}), frozenset({1, 4, 5}), frozenset({2, 4, 5}), frozenset({3, 4, 5})))
```

3. Labelled vs. Unlabelled Trees

According to Cayley's formula, there are n^{n-2} **labelled trees** on the n-point vertex set $X = \{0, 1, 2, \ldots, n-1\}$.

The **isomorphism types** of trees, aka **unlabelled trees**, on n vertices are much harder to count: there is no closed formula for the number of unlabelled trees.

Using the concept of group actions, we have already determined the **orbits** of the labelled trees on 4 nodes under all permutations of those 4 nodes. As a result we have seen that there are (only) 2 unlabelled trees on 4 points, one orbit of length 4 and one of length 12, accounting for a total of $4^{4-2} = 16$ labelled trees.

The same method yields unlabelled trees on more than 4 points, as follows. Recall the tools that have been used in the case $n = 4$.

- We have represented permutations on n points as `python` tuples of length n, in particular the transposition swapping points i and $i+1$:

In [17]:
```
def transposition(n, i):
    t = list(range(n))
    t[i], t[i+1] = t[i+1], t[i]
    return tuple(t)
```

- We have checked equality of graphs by checking whether their node sets and their edge sets are the same:

In [18]:
```
def is_subgraph(G, H):
    for x in G:
        if not H.has_node(x):
            return False
    for e in G.edges:
        if not H.has_edge(*e):
            return False
    return True

def are_equal_as_graphs(G, H):
    return is_subgraph(G, H) and is_subgraph(H, G)
```

- We have set up a BFS over the implicit action graph spanned by the action of the symmetric group as an **orbit algorithm**:

In [19]:
```
def orbit(gens, x, action, equals):
    Q = [x]
    for y in Q:
        for a in gens:
            z = action(a, y)
            w = next((v for v in Q if equals(v, z)), None)
            if w is None:
                Q.append(z)
    return Q
```

- And we have formulated the relabelling of nodes as a group action:

In [20]:
```
def relabel(a, G):
    mapping = dict(enumerate(a))
    return nx.relabel_nodes(G, mapping)
```

- Also, we'll use the `is_isomorphic` function provided by `networkx` to test isomorphism of graphs.

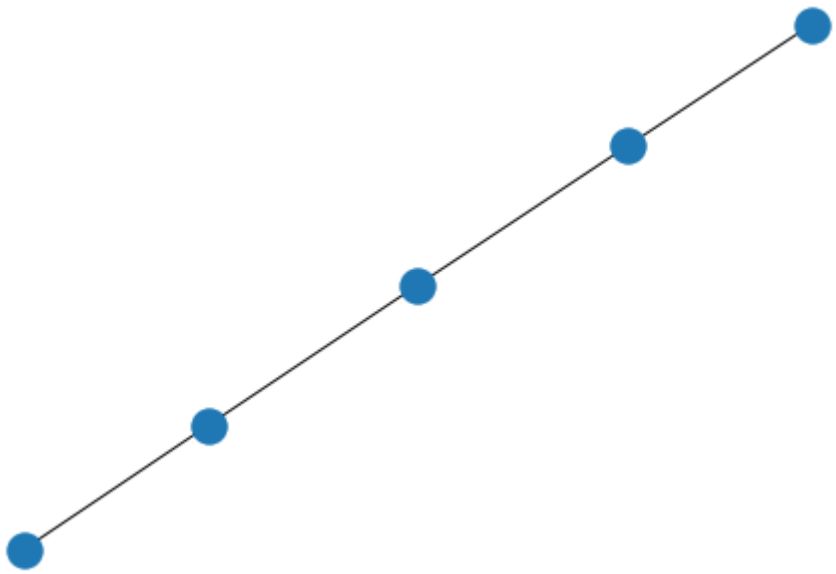
With all of this in place, we can systematically list all unlabelled trees on $n = 5$ nodes:

- Start by setting `n` to 5, listing the four generating transpositions of the symmetric group S_5 as `swaps`, and initializing a list of `orbits`, for now as an empty list:

In [21]:
```
n = 5
swaps = [transposition(n, k) for k in range(n-1)]
orbits = []
```

- Pick a random tree `t` on n points and draw it:

In [22]:
```
t = nx.random_tree(n)
nx.draw(t)
```



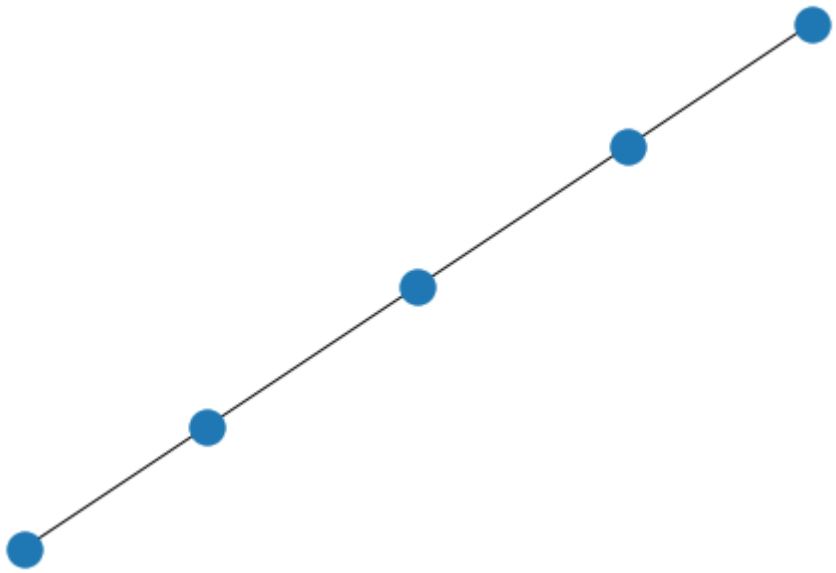
- Compute the orbit of `t` under the generating permutations `swaps` with respect to

the `relabel` action, append the orbit to the list `orbits` and check its length:

In [23]:
```
orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

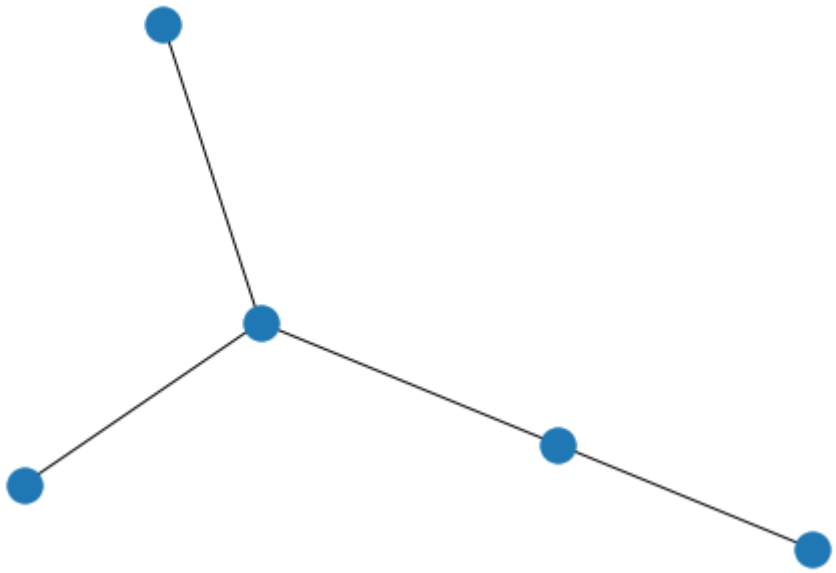
Out[23]: 60

In [24]:
```
nx.draw(orbits[0][5])
```



- Pick another random tree `t`; in case it is isomorphic to the old tree `t` (which now is `orbits[0][0]`) pick again, and keep picking until a tree `t` is found that's not isomorphic to the old one; draw `t` - it should have a different shape:

In [25]:
```
t = nx.random_tree(n)
while nx.is_isomorphic(t, orbits[0][0]):
    t = nx.random_tree(n)
nx.draw(t)
```



- Compute its orbit and append it to the list `orbits` :

In [26]:
```
orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[26]: 60

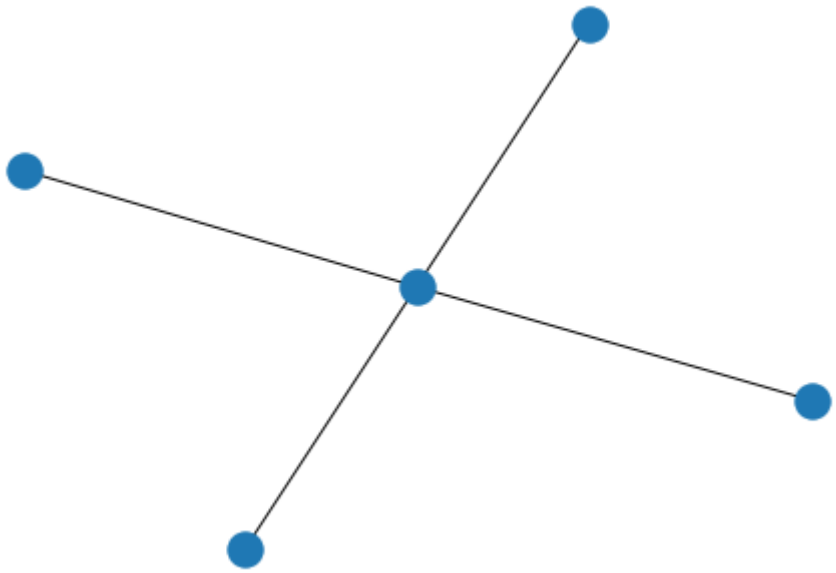
- How many labelled trees have we found so far?

In [27]:
```
sum(len(o) for o in orbits)
```

Out[27]: 120

- That's less than 125, so we keep searching by repeating the above steps - first keep picking random trees until one is found that is not isomorphic to **any** of the previously found ones (which now are found as `o[0] for o in orbits` , the first tree in each of the orbits found so far):

In [28]:
```
t = nx.random_tree(n)
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)

nx.draw(t)
```



- Then append its orbit to the list `orbits` :

In [29]:
```
orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[29]: 5

- Have we found them all?

In [30]:
```
sum(len(o) for o in orbits)
```

Out[30]: 125

In [31]:
```
len(orbits)
```

Out[31]: 3

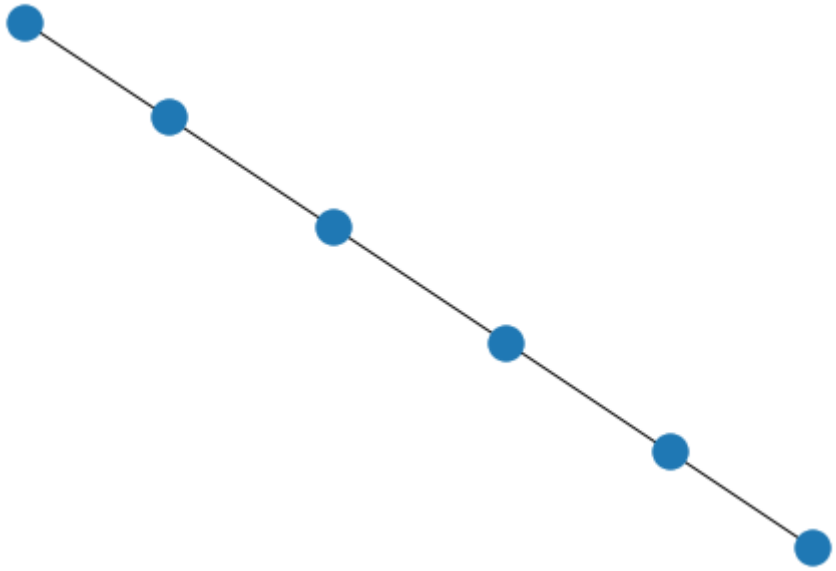
- Yes, all 125! So that's 3 unlabelled trees on 5 points, with orbits of length 60, 60 and 5.

How many unlabelled trees are there $n = 6$ points? Enumerate the **orbits of labelled trees** on 6 points as above, by following these steps:

1. Start by setting `n` to 6, `swaps` to the list of 5 generating transpositions of the symmetric group S_6, and `orbits` to the empty list.
2. Pick a random tree `t` and draw it.
3. Compute the orbit `orb` of `t` under the generating permutations `swaps`, append `orb` to the list `orbits` and check the length of `orb`.
4. Repeat the following steps until the sum of the orbit lengths in `orbits` equals $6^{6-2} = 1296$:
5. Keep picking random trees on 6 points until a tree `t` is found that is not isomorphic to any of the previous ones, then draw `t` (is it a new shape?).
6. Compute the orbit `orb` of `t` under the generating permutations `swaps`, append `orb` to the list `orbits` and check the length of `orb`.
7. Check the sum of the orbit lengths in `orbit`. STOP if the desired number is reached.
8. How many unlabelled trees on 6 points did you find?
9. How big are their **automorphism groups**? (Use the Orbit-Stabilizer Theorem.)

In [32]:
```
#3.1
n = 6
swaps = [transposition(n, k) for k in range(n-1)]
orbits = []

#3.2
t = nx.random_tree(n)
nx.draw(t)

#3.3
orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

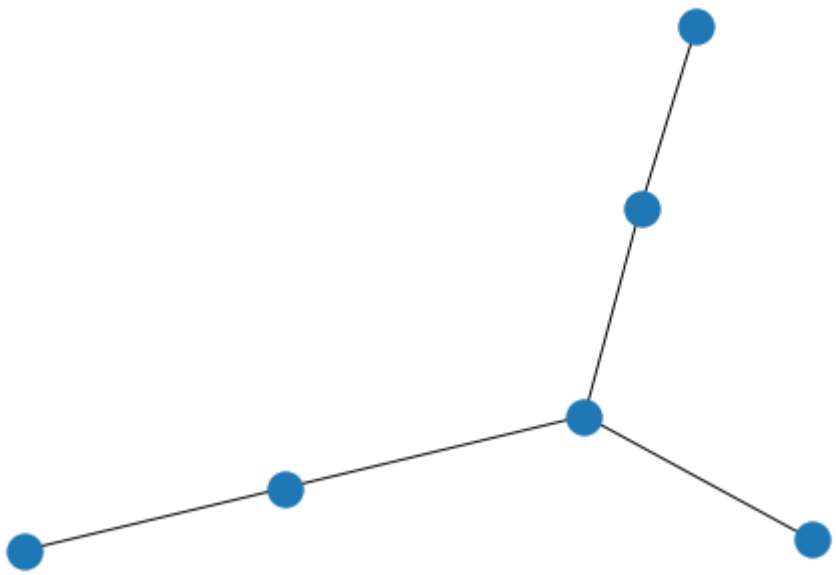
Out[32]: 360



In [33]:

```
#3.4-3.7
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)
nx.draw(t)

orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[33]: 360

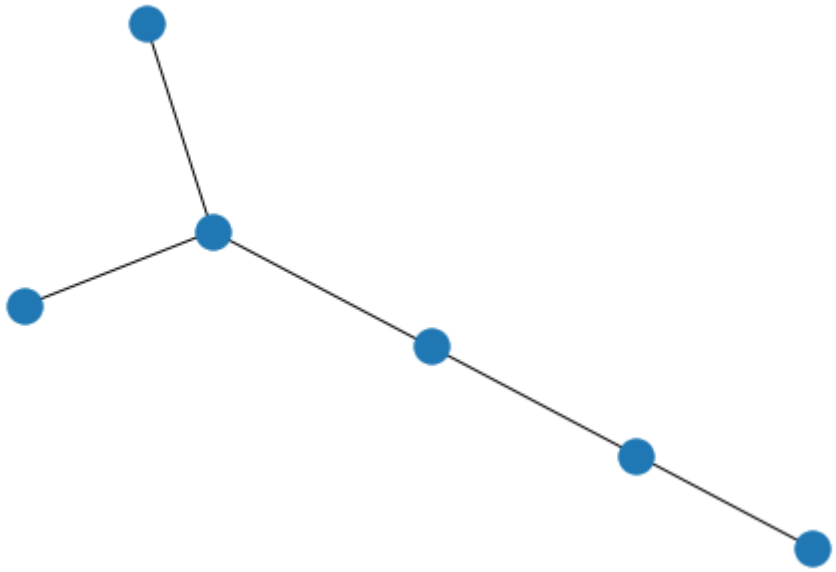


A new shape is found, and its orbit has been appended to orbits.

In [34]:

```
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)
nx.draw(t)

orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[34]: 360

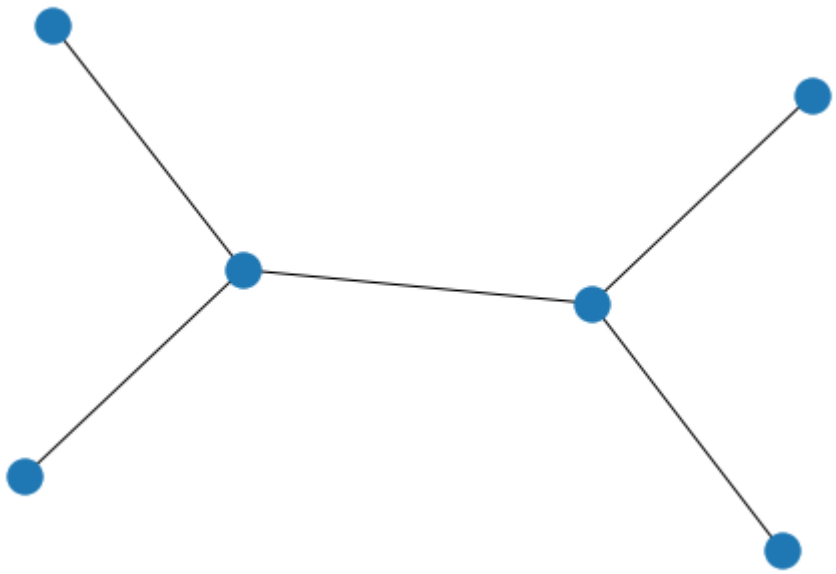


Another new shape is found, and its orbit has been appended to orbits.

In [35]:

```
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)
nx.draw(t)

orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

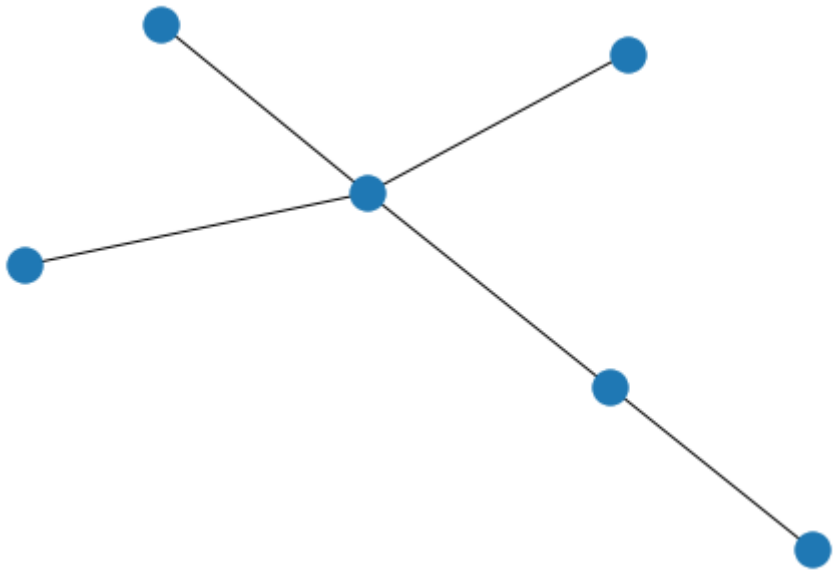
Out[35]: 90



Another new shape is found, and its orbit has been appended to orbits.

In [36]:
```
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)
nx.draw(t)

orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[36]: 120



Another new shape is found, and its orbit has been appended to orbits.

In [37]:
```
while any(nx.is_isomorphic(t, o[0]) for o in orbits):
    t = nx.random_tree(n)
nx.draw(t)

orb = orbit(swaps, t, relabel, are_equal_as_graphs)
orbits.append(orb)
len(orb)
```

Out[37]: 6

In [38]:
```
sum(len(o) for o in orbits)
```

Out[38]: 1296

With the sum of the orbit lengths now $6^{6-2} = 1296$: we have found all the unlabelled trees for 6 points!

In [39]:
```
#3.8
len(orbits)
```

Out[39]: 6

We found 6 different unlabeled trees on 6 points.

Using the Orbit-Stabilizer Theorem we can calculate the size of their automorphism groups by dividing the size of the group S6 by the size of each of their orbits.

In [40]:
```
#3.9
# 6! = 720
720/len(orbits[0])
```

Out[40]: 2.0

In [41]:
```
720/len(orbits[1])
```

Out[41]: 2.0

In [42]:
```
720/len(orbits[2])
```

Out[42]: 2.0

In [43]:
```
720/len(orbits[3])
```

Out[43]: 8.0

In [44]:
```
720/len(orbits[4])
```

Out[44]: 6.0

In [45]:
```
720/len(orbits[5])
```

Out[45]: 120.0

In [46]:
```
#3.9
#Order of S6
order = 6*5*4*3*2*1
automorphisms = [order/len(o) for o in orbits]
```

In [47]:
```
automorphisms
```

Out[47]: [2.0, 2.0, 2.0, 8.0, 6.0, 120.0]

4. A Collaborations Network

The **social graph** of a node x in a (social) network is the **induced subgraph** on the set of friends of x (that is the graph which has **only** the friends of x as its vertices, and between them all the edges from the original network; x itself is **not** a node in its social graph). The **clustering coefficient** of x is the density $m\binom{n}{2}^{-1}$ of the social graph of x, the proportion of its actual number of edges, m, in the set of potential edges, of size $\binom{n}{2} = \frac{1}{2}n(n-1)$, where n is its number of vertices.

MathSciNet describes the social network of mathematical researchers defined by collaboration.

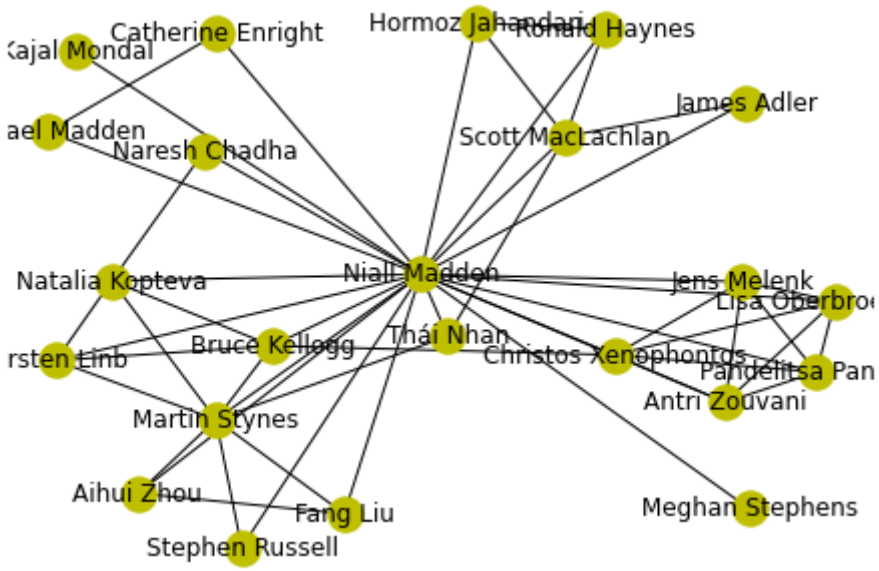
1. **Pick** a (local) mathematician with at least 10 friends (i.e., co-authors), determine their social graph.

2. Hence compute their clustering coefficient.

(**Note** that access to these data is free, but only **on campus** or through the **library** website.)

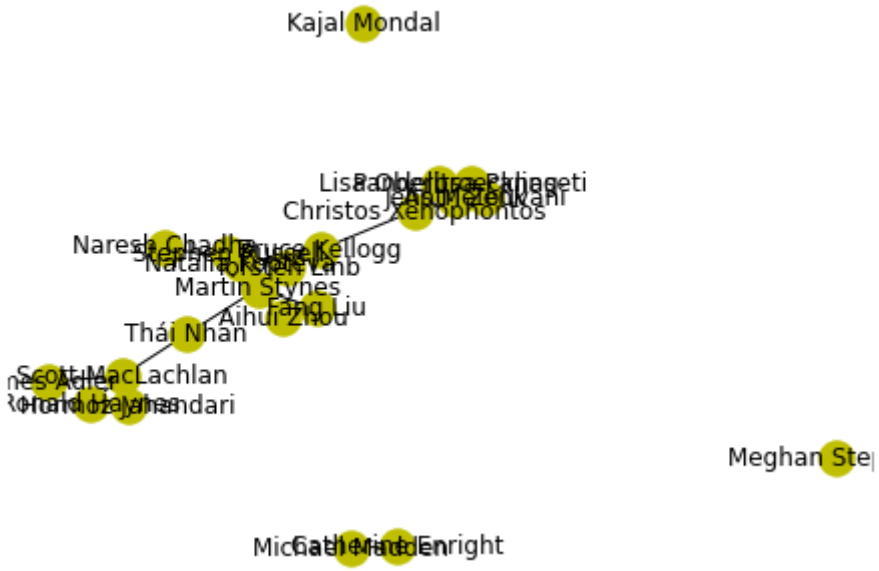
In [48]:

```
#4.1

G = nx.Graph()

G.add_node('Niall Madden')

friends = ['James Adler', 'Naresh Chadha', 'Catherine Enright',
           'Ronald Haynes', 'Hormoz Jahandari',
          'Bruce Kellogg', "Natalia Kopteva",
           'Torsten Linb', 'Fang Liu', 'Scott MacLachlan',
           'Michael Madden',
          'Jens Melenk','Kajal Mondal',
           'Thái Nhan', 'Lisa Oberbroeckling', 'Pandelitsa Panaseti',
          'Stephen Russell', 'Meghan Stephens',
           'Martin Stynes', 'Christos Xenophontos', 'Aihui Zhou',
           'Antri Zouvani']

G.add_edges_from([('Niall Madden', i) for i in friends])

G.add_edges_from([('James Adler', 'Scott MacLachlan')])

G.add_edges_from([('Naresh Chadha',"Natalia Kopteva")])

G.add_edges_from([('Catherine Enright', 'Michael Madden')])

G.add_edges_from([('Ronald Haynes', friend) for friend in ['Hormoz Jahandar

G.add_edges_from([('Hormoz Jahandari', 'Scott MacLachlan')])

G.add_edges_from([('Bruce Kellogg', friend) for friend in ["Natalia Kopteva

G.add_edges_from([('Natalia Kopteva', friend) for friend in ['Torsten Linb'

G.add_edges_from([('Torsten Linb', 'Martin Stynes')])

G.add_edges_from([('Fang Liu', friend) for friend in ['Aihui Zhou', 'Martin

G.add_edges_from([('Scott MacLachlan', 'Thái Nhan')])

G.add_edges_from([('Jens Melenk', friend) for friend in['Lisa Oberbroecklin

G.add_edges_from([('Thái Nhan', 'Martin Stynes')])

G.add_edges_from([('Thái Nhan', 'Martin Stynes')])

G.add_edges_from([('Lisa Oberbroeckling', friend) for friend in ['Antri Zou

G.add_edges_from([('Pandelitsa Panaseti', friend) for friend in ['Antri Zou

G.add_edges_from([('Stephen Russell', 'Martin Stynes')])

G.add_edges_from([('Stephen Russell', 'Martin Stynes')])

G.add_edges_from([('Martin Stynes', 'Aihui Zhou')])

G.add_edges_from([('Christos Xenophontos', 'Antri Zouvani')])

nx.draw(G, **opts)
```



In [49]:

```
social_graph = nx.subgraph(G, friends)
nx.draw(social_graph, **opts)
```



In [50]:

```
#4.2 Clustering coefficient
print("Edges of social graph:", len(social_graph.edges()))
```

```
Edges of social graph: 29
```

In [51]:

```
print("Nodes of social graph:", len(social_graph.nodes()))
```

```
Nodes of social graph: 22
```

In [52]:

```
print("Possible edges of social graph:", 1/2 * 22*21)
```

```
Possible edges of social graph: 231.0
```

In [53]:

```
print("The clustering coefficient is:", nx.density(social_graph))
```

```
The clustering coefficient is: 0.12554112554112554
```

In [54]:
```
nx.clustering(G, 'Niall Madden')
```

Out[54]:
```
0.12554112554112554
```